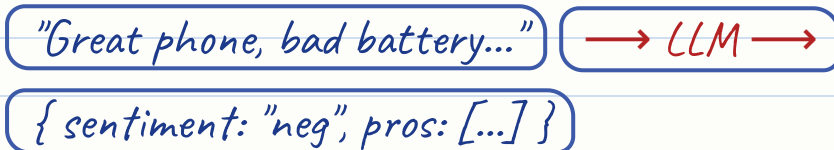


Structured Output

getting dicts / objects out of an LLM instead of plain text

Why do we need it?

- LLM normally returns **free text**, which is fine for humans but useless for code.
- Apps need **machine-readable data** (dict / JSON) to store in a DB, call an API, or pass to the next step.
- Agents need it too: a **tool call** is basically structured output (tool name + args).



Two ways to get it

- **Model supports it natively** (OpenAI, Claude, Gemini...) → `model.with_structured_output(schema)`
- **Model doesn't** (many open-source HF models) → **Output Parsers** (page 2)

with_structured_output()

You pass a **schema** describing the shape you want. 3 ways to write the schema:

① TypedDict

- Just Python type hints: says what keys + types a dict should have.
- `Annotated[type, "description"]` adds a hint so the LLM knows what to fill.
- `Optional[...]` = key may be missing. `Literal["pos", "neg"]` = only these values.
- **No validation at runtime!** If the LLM sends the wrong type, nothing stops it.

TypedDict example

```
from typing import TypedDict, Annotated, Optional, Literal

class Review(TypedDict):
    key_themes: Annotated[list[str], "main themes in the review"]
    summary: Annotated[str, "brief summary"]
    sentiment: Annotated[Literal["pos", "neg"], "overall sentiment"]
    pros: Annotated[Optional[list[str]], "pros, if any"]

structured_model = model.with_structured_output(Review)
result = structured_model.invoke(review_text)
result["summary"] # returns a dict
```

② Pydantic (most used ★)

- Data validation library: checks types at runtime and raises an error if wrong.
- Supports defaults, Optional, constraints like Field(gt=0, lt=10), EmailStr.
- Type coercion: "32" becomes 32 for an int field.
- Field(description=...) works like Annotated: a hint for the LLM.
- Result is a Pydantic object → result.summary, convert with .model_dump() / .model_dump_json().

Pydantic example

```
from pydantic import BaseModel, Field
from typing import Optional, Literal

class Review(BaseModel):
    summary: str = Field(description="brief summary")
    sentiment: Literal["pos", "neg"] = Field(description="sentiment")
    rating: Optional[float] = Field(default=None, gt=0, lt=10)

result = model.with_structured_output(Review).invoke(review_text)
result.summary # attribute access
result.model_dump() # → dict
```

③ JSON Schema

- Plain JSON dict describing the structure: language-agnostic.
- Use when the project mixes Python + JS/other backends that share one schema.
- Result is a dict.

JSON Schema example

```
review_schema = {
    "title": "Review",
    "type": "object",
    "properties": {
        "summary": {"type": "string", "description": "brief summary"},
        "sentiment": {"type": "string", "enum": ["pos", "neg"]}
    },
    "required": ["summary", "sentiment"]
}

model.with_structured_output(review_schema).invoke(review_text)
```

Which one when?

Schema	Use when...	Validates?	Returns
TypedDict	quick, Python-only, trust the model	✗	dict
Pydantic	need validation / defaults / constraints	✓	object
JSON Schema	multi-language project	✓*	dict

* JSON schema constrains the model's output on supported providers; no Python-side checks like Pydantic.

The `method` argument

- `method="function_calling"`: uses the model's tool-calling to fill the schema.
- `method="json_mode"`: model just promises valid JSON; describe the keys in your prompt.
- `method="json_schema"`: provider-native structured outputs (strictest, where supported).
- Not every provider supports every method, so check the integration docs.

Output Parsers

turn raw LLM text into structured data, works with ANY model

- Parsers are classes in `langchain_core.output_parsers`.
- They are **Runnables**, so they slot straight into chains: `prompt | model | parser`
- Most give you `parser.get_format_instructions()`, text you put in the prompt telling the LLM how to format.

① StrOutputParser

- Simplest: takes the `AIMessage` and returns just `.content` as a string.
- Real use: **chaining** LLM calls, where the output of one becomes input of the next.

detailed report → 5-line summary

```
from langchain_core.output_parsers import StrOutputParser
parser = StrOutputParser()

chain = template1 | model | parser | template2 | model | parser
chain.invoke({"topic": "black hole"})
```

② JsonOutputParser

- Forces the LLM to reply in JSON and returns a **dict**.
- **Catch:** you can't control the schema; the LLM decides the keys.

JSON parser + format instructions

```
from langchain_core.output_parsers import JsonOutputParser
from langchain_core.prompts import PromptTemplate

parser = JsonOutputParser()
template = PromptTemplate(
    template="Give 5 facts about {topic}\n{format_instruction}",
    input_variables=["topic"],
    partial_variables={"format_instruction": parser.get_format_instructions()},
)
chain = template | model | parser
chain.invoke({"topic": "black hole"}) # → dict
```

partial_variables = filled at template creation time (not at invoke time).

③ StructuredOutputParser

- Define the keys with `ResponseSchema(name=..., description=...)`, so the schema is enforced.
- **Catch:** no data validation (age = "thirty" would pass).
- Older API: lives in `langchain.output_parsers` (moved to `langchain_classic` in newer versions).

keys enforced, types not

```
from langchain.output_parsers import StructuredOutputParser, ResponseSchema

schema = [
    ResponseSchema(name="fact_1", description="Fact 1 about the topic"),
    ResponseSchema(name="fact_2", description="Fact 2 about the topic"),
]
parser = StructuredOutputParser.from_response_schemas(schema)
```

④ PydanticOutputParser ★

- Schema enforced + type validation + constraints, the best of both.
- Returns a Pydantic object.

fictional person generator

```
from langchain_core.output_parsers import PydanticOutputParser
from pydantic import BaseModel, Field

class Person(BaseModel):
    name: str = Field(description="name of the person")
    age: int = Field(gt=18, description="age of the person")
    city: str = Field(description="city they belong to")

parser = PydanticOutputParser(pydantic_object=Person)
template = PromptTemplate(
    template="Generate name, age and city of a fictional {place} person\n{format_instruction}",
    input_variables=["place"],
    partial_variables={"format_instruction": parser.get_format_instructions()},
)
chain = template | model | parser
chain.invoke({"place": "Indian"})
```

Cheat sheet: 4 parsers

Parser	Gives	Keys enforced?	Types validated?
StrOutputParser	string	n/a	n/a
JsonOutputParser	dict	✗	✗
StructuredOutputParser	dict	✓	✗
PydanticOutputParser	object	✓	✓

Big picture

`with_structured_output()` = model does the work natively (more reliable).

Output parsers = prompt instructions + parse after, so they work with any model, even ones without native support.

Mistakes to avoid 📝

- Forgetting `{format_instruction}` in the prompt, so the parser has nothing to parse.
- Assuming TypedDict validates; it doesn't.
- Pydantic v2: use `.model_dump()`, not the old `.dict()`.
- JsonOutputParser ≠ fixed schema. Need fixed keys? Use Pydantic.

Databricks exam angle

- Expect questions on getting **reliable JSON** from an LLM: format instructions in the prompt, schema-constrained output, validating before passing downstream.
- Tool calling in agents = structured output under the hood.

next: Chains → Runnables → Document loaders